

Lab 1: Experiencing Blocking (Row-Level Lock)

The image displays four sequential screenshots of a PostgreSQL client interface, likely pgAdmin, showing the execution of a series of SQL queries. The interface includes a top toolbar with icons for file operations, a query editor, and a results/output pane at the bottom.

Top Left Screenshot: The query editor shows a multi-line SQL script:

```
1 --Session A (Staff 1)
2 -- Step 1
3 BEGIN;
4 UPDATE film SET rental_rate = 4.99
5 WHERE film_id = 1;
6 -- Lúc này, Session A đang giữ X-Lock trên film_id = 1
```

The 'Messages' tab in the output pane shows:

```
UPDATE 1
Query returned successfully in 97 msec.
```

The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.097 CRLF Ln 3, Col 1'.

Top Right Screenshot: The query editor shows the continuation of the script:

```
1 --Session B (Staff 2)
2 -- Step 2
3 BEGIN;
4 UPDATE film SET rental_rate = 5.99
5 WHERE film_id = 1;
6 -- KẾT QUẢ: Lệnh này sẽ bị BLOCK NGAY LẬP TỨC và ở trạng
```

The 'Messages' tab is empty, displaying a message: 'No data output. Execute a query to get output.'

Bottom Left Screenshot: This screenshot is identical to the top-left one, showing the first query and its successful execution message.

Bottom Right Screenshot: The query editor shows the same script as the previous screenshots. The 'Messages' tab is empty. The status bar at the bottom indicates 'Total rows: Waiting for the query to complete... 00:00:03.227 CRLF Ln 3, Col 1', indicating a timeout or wait state.

Lab 2: Replicating Non-repeatable Read

dvdrental/postgres@PostgreSQL 18

No limit

E

Query

Query History

1 -- session A

2 -- Step 1

3 BEGIN;

4 SELECT rental_rate FROM film

5 WHERE film_id = 10;

Data Output

Messages

Notifications

SQL

Showing rows: 1
to 1

Page No: 1 of 1

rental_rate
numeric(4,2)

1 4.99

Total rows: 1 Query complete 00:00:00.200 CRLF Ln 3, Col 1

dvdrental/postgres@PostgreSQL 18

No limit

E

Query

Query History

1

Data Output

Messages

Notifications

No data output. Execute a query to get output.

Total rows: CRLF Ln 1, Col 1

dvdrental/postgres@PostgreSQL 18

No limit

Query

Query History

1

-- session A

2

-- Step 1

3

BEGIN;

4

SELECT rental_rate FROM film

5

WHERE film_id = 10;

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

SQL

rental_rate	numeric (4,2)
1	4.99

Total rows: 1

Query complete 00:00:00.200

CRLF

Ln 3, Col 1

dvdrental/postgres@PostgreSQL 18

No limit

Query

Query History

1

-- session B

2

-- Step 2

3

BEGIN;

4

UPDATE film SET rental_rate = 0.99

5

WHERE film_id = 10;

6

7

COMMIT;

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 34 msec.

✓ Query returned successfully in 34 msec. ✕

Total rows:

Query complete 00:00:00.034

CRLF

Ln 3, Col 1

dvdrental/postgres@PostgreSQL 18

No limit

Query

Query History

1

-- session A

2

-- Step 1

3

BEGIN;

4

SELECT rental_rate FROM film

5

WHERE film_id = 10;

6

7

-- Step 3

8

SELECT rental_rate FROM film

9

WHERE film_id = 10;

10

11

12

COMMIT;

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

SQL

rental_rate	numeric (4,2)
1	0.99

Total rows: 1

Query complete 00:00:00.042

CRLF

Ln 8, Col 1

dvdrental/postgres@PostgreSQL 18

No limit

Query

Query History

1

-- session B

2

-- Step 2

3

BEGIN;

4

UPDATE film SET rental_rate = 0.99

5

WHERE film_id = 10;

6

7

COMMIT;

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 34 msec.

Total rows:

Query complete 00:00:00.034

CRLF

Ln 3, Col 1

Lab 3: Creating a Deadlock

No limit

E

Query

Query History

1

-- session A

2

-- Step 1

3

BEGIN;

4

UPDATE film SET rental_rate = 2.99

5

WHERE film_id = 100;

6

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 49 msec.

Total rows:

Query complete 00:00:00.049

CRLF

Ln 3, Col 1

No limit

E

Query

Query History

1

-- session B

2

-- Step 2

3

BEGIN;

4

UPDATE film SET rental_rate = 2.99

5

WHERE film_id = 200;

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 32 msec.

Total rows:

Query complete 00:00:00.032

CRLF

Ln 3, Col 1

No limit

E

Query

Query History

1

-- session A

2

-- Step 1

3

BEGIN;

4

UPDATE film SET rental_rate = 2.99

5

WHERE film_id = 100;

6

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 49 msec.

Total rows:

Query complete 00:00:00.049

CRLF

Ln 3, Col 1

No limit

E

Query

Query History

1

-- session B

2

-- Step 2

3

BEGIN;

4

UPDATE film SET rental_rate = 2.99

5

WHERE film_id = 200;

Data Output

Messages

Notifications

UPDATE 1

Query returned successfully in 32 msec.

Total rows:

Query complete 00:00:00.032

CRLF

Ln 3, Col 1

[illegible]

ERROR: deadlock detected

Detail: Process 25568 waits for ShareLock on transaction 1043; blocked by process 11792.
Process 11792 waits for ShareLock on transaction 1044; blocked by process 25568.
Hint: See server log for query details.
Context: while locking tuple (5,12) in relation "film"

Lab 4: Preventing Race Conditions (FOR UPDATE)

The image shows two side-by-side PostgreSQL query editors. Both are connected to 'dvdrental/postgres@PostgreSQL 18'.

Left Editor (Session A):

- Query 1: `--Session A (Customer 1)`
- Query 2: `-- Step 1`
- Query 3: `BEGIN;`
- Query 4: `SELECT * FROM inventory`
- Query 5: `WHERE inventory_id = 10 FOR UPDATE;`

Data Output (Left Editor):

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2006-02-15 10:09:17

Right Editor (Session B):

- Query 1: `--Session B (Customer 2)`
- Query 2: (Empty)

Data Output (Right Editor):

No data output. Execute a query to get output.

Status Bars:

- Left Editor: Total rows: 1 | Query complete 00:00:00.095 | CRLF | Ln 2, Col 1
- Right Editor: Total rows: | Query complete 00:00:01.028 | CRLF | Ln 2, Col 1

QueryQuery History

1--Session A (Customer 1)

2-- Step 1

3BEGIN;

4SELECT * FROM inventory

5WHERE inventory_id = 10 FOR UPDATE;

6

7-- Step 3

8INSERT INTO rental

9(rental_date, inventory_id, customer_id, staff_id)

10VALUES (NOW(), 10, 2, 1);

11

12COMMIT;

Data OutputMessagesNotifications

Showing rows: 1 to 1

Page 1 of 1

No: 1

SQL

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2	2006-02-15 10:09:17

Total rows: 1

Query complete 00:00:00.041

CRLF

Ln 3, Col 7

QueryQuery History

1--Session B (Customer 2)

2-- Step 2

3BEGIN;

4SELECT * FROM inventory

5WHERE inventory_id = 10 FOR UPDATE;

Data OutputMessagesNotifications

Total rows: 1

Waiting for the query to complete... 00:00:02.225

CRLF

Ln 2, Col 1

dvdrental/postgres@PostgreSQL 18

Showing rows: 1 to 1

Page 1 of 1

No: 1

SQL

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2	2006-02-15 10:09:17

Total rows: 1

Query complete 00:00:00.046

CRLF

Ln 7, Col 1

dvdrental/postgres@PostgreSQL 18

Showing rows: 1 to 1

Page 1 of 1

No: 1

SQL

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2	2006-02-15 10:09:17

Total rows: 1

Query complete 00:00:16.168

CRLF

Ln 2, Col 1

1--Session A (Customer 1)

2-- Step 1

3BEGIN;

4SELECT * FROM inventory

5WHERE inventory_id = 10 FOR UPDATE;

6

7-- Step 3

8INSERT INTO rental

9(rental_date, inventory_id, customer_id, staff_id)

10VALUES (NOW(), 10, 2, 1);

11

12COMMIT;

COMMIT

Query returned successfully in 46 msec.

1--Session B (Customer 2)

2-- Step 2

3BEGIN;

4SELECT * FROM inventory

5WHERE inventory_id = 10 FOR UPDATE;

Showing rows: 1 to 1

Page 1 of 1

No: 1

SQL

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2	2006-02-15 10:09:17

Total rows: 1

Query complete 00:00:16.168

CRLF

Ln 2, Col 1